Project Report - Task 3

Data Storage Paradigms, IV1351

2025/11

Group 27 project member:
[Sam Serbouti, serbouti@kth.se] (individual work for this task)

1 Introduction

This report presents the implementation of a layered Java application that accesses a PostgreSQL database to manage university courses and teaching allocations. The focus is on correct ACID transaction handling, enforcement of business rules in the database, and a clean separation of concerns between view, controller, service, and integration (DAO) layers. The program:

- computes planned and actual teaching costs for course instances
- updates course instances when the number of students changes
- allocates and deallocates teaching activities for teachers
- supports adding new teaching activities such as *Exercise*

The implementation builds directly on the relational schema and OLAP queries developed in the earlier tasks and reuses the existing workload view for planned course hours as well as a trigger for excessive teacher workload.

The project member aims at a higher grade and implemented a GUI that uses Java Swing and Flatlaf for a modern look.

2 Literature Study

I gained knowledge on the design and implementation part of the task by the course material on database applications, in particular the JDBC bank example and its explanation of layered architecture, transaction handling, and the DAO/DTO patterns. I also read a GeekfsForGeeks article on DAOs. The associated tips-and-tricks document for the seminar provided concrete coding help for JDBC, including transaction management and the use of `SELECT ...FOR UPDATE` in read/modify/write scenarios. I also used a coding tech room article for information on how to use Flatlaf.

3 Method

Development was carried out in an Visual Studio Code with support for Maven projects and Swing for the simple graphical user interface.

The program was structured in layers following the MVC and Layer patterns. The view layer consists of Swing panels such as `CostPanel`, `AllocationPanel`, and `ActivityPanel`, which are responsible only for user interaction and presenting results. The controller layer (`CourseController`, `AllocationController`, `ActivityController`) exposes application use cases to the views and delegates all transactional work to corresponding service classes.

The integration layer is split into two parts:

1. A `DatabaseManager` handles JDBC driver configuration, connection creation with auto-commit disabled, and explicit commit and rollback.
2. Data access objects (`CourseDAO`, `AllocationDAO`, `ActivityDAO`) encapsulate all SQL statements and provide CRUD-style operations and aggregations without embedding any business rules.

At the database level, I extended the schema from the previous task. A `v_full_course_workload` view reuses and formalizes the earlier planned-hours query, while triggers on `employee_activity` enforce the maximum number of course instances a teacher may handle per study period and year.

I had to add a column to the cross reference table `employee_activity` to have a way of tracking planned vs actually accomplished hours.

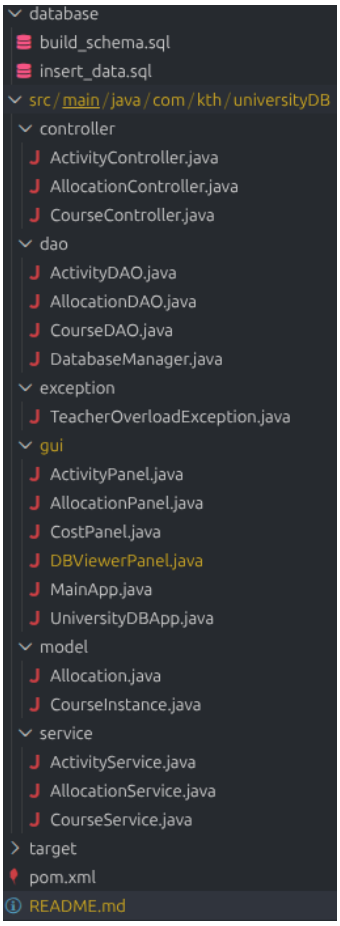
4 Result

You can see the final program in my personal public github repo.

4.1 Computing teaching cost

For a selected course instance in a given year and period, the program reports both planned and actual teaching cost. Planned hours are obtained from the view `v_full_course_workload`, which sums weighted hours for all planned activities and adds derived examination and administration hours based on the number of students and course credits. Actual hours are computed by summing the `allocated_hours` stored in `employee_activity` for all activities belonging to the instance. The service layer multiplies both planned and actual hours by an average salary per hour and divides by 1000 to display costs in kSEK.

Users can compute the cost for an existing course instance via the cost panel and then re-run the computation after updating the number of students, as described below. Sample output from the running program is included in the appendix of the full report.


database
build_schema.sql
insert_data.sql
src / main / java / com / kth / universityDB
controller
ActivityController.java
AllocationController.java
CourseController.java
dao
ActivityDAO.java
AllocationDAO.java
CourseDAO.java
DatabaseManager.java
exception
TeacherOverloadException.java
gui
ActivityPanel.java
AllocationPanel.java
CostPanel.java
DBViewerPanel.java
MainApp.java
UniversityDBApp.java
model
Allocation.java
CourseInstance.java
service
ActivityService.java
AllocationService.java
CourseService.java
target
pom.xml
README.md

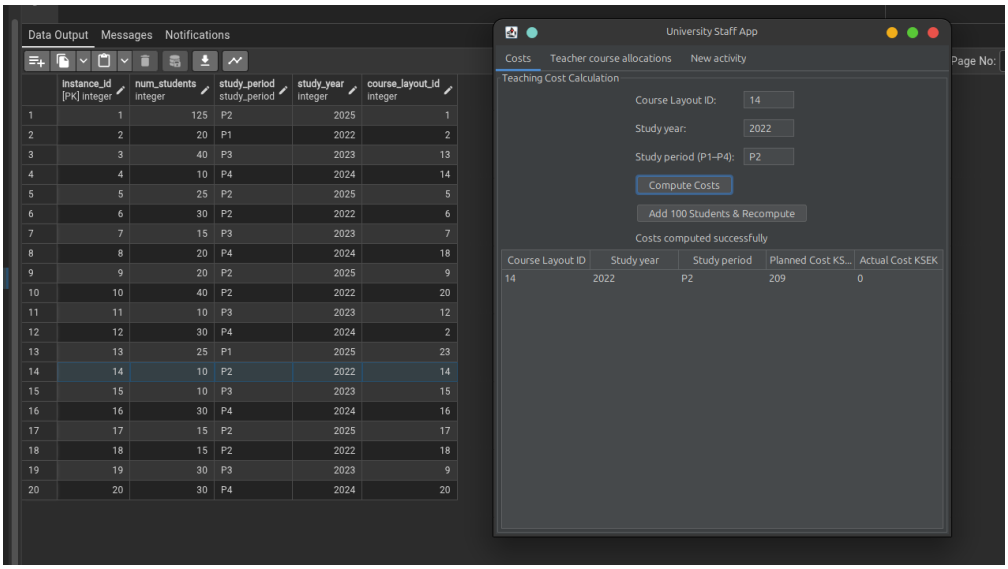


Figure 1: Computing the cost of a course

4.2 Updating course instance and recomputing cost

The application allows the user to select a specific course layout, year, and period, increase the number of registered students in the corresponding instance by 100, and recompute both planned and actual cost. This sequence is implemented as a single transaction that locks the selected course instance row with `SELECT ...FOR UPDATE`, reads the current number of students, updates the value, recomputes hours and costs, and then commits. The result panel prints both the previous and updated cost values so that the impact of the increased number of students is visible.

4.3 Allocating and deallocating teaching loads

Teachers can be allocated to planned activities by entering their employment ID, a course layout, year, period, a planned activity ID, and the number of hours to allocate. The allocation panel sends this information to the allocation controller, which delegates to the allocation service. The service verifies that the targeted course instance exists and then inserts a new row into `employee_activity`. A database trigger checks the number of distinct course instances already associated with that teacher in the given period and year and raises an error if the teacher would exceed four instances. The service catches this error and reports it back to the GUI as a domain-specific exception.

Deallocation is implemented by issuing a `DELETE` on `employee_activity` for the given teacher and planned activity. The program demonstrates at least one successful allocation, one successful deallocation, and one case where an attempted allocation fails because the teacher already has four course instances in that study period and year.

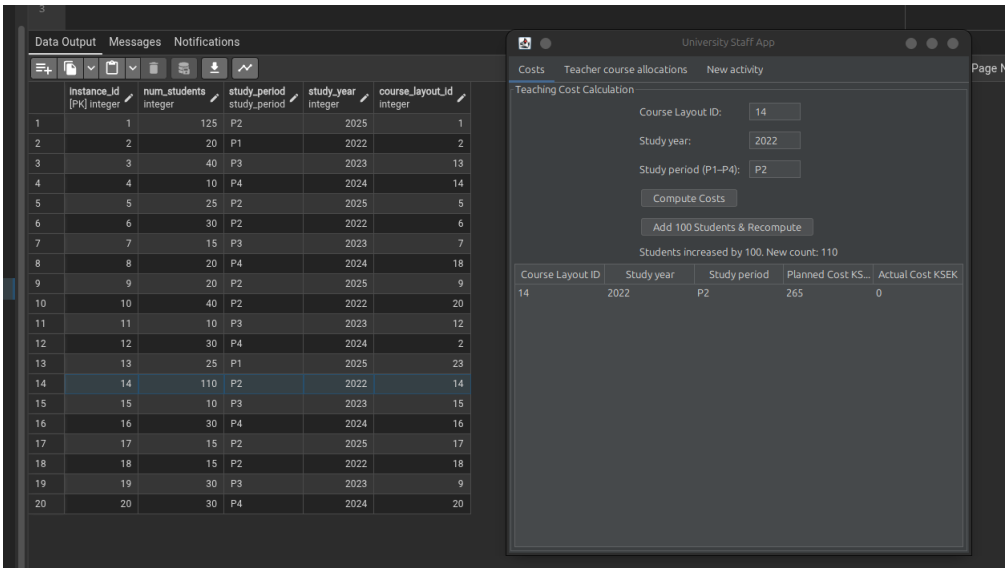


Figure 2: Addind 100 students to a course and computing its final cost

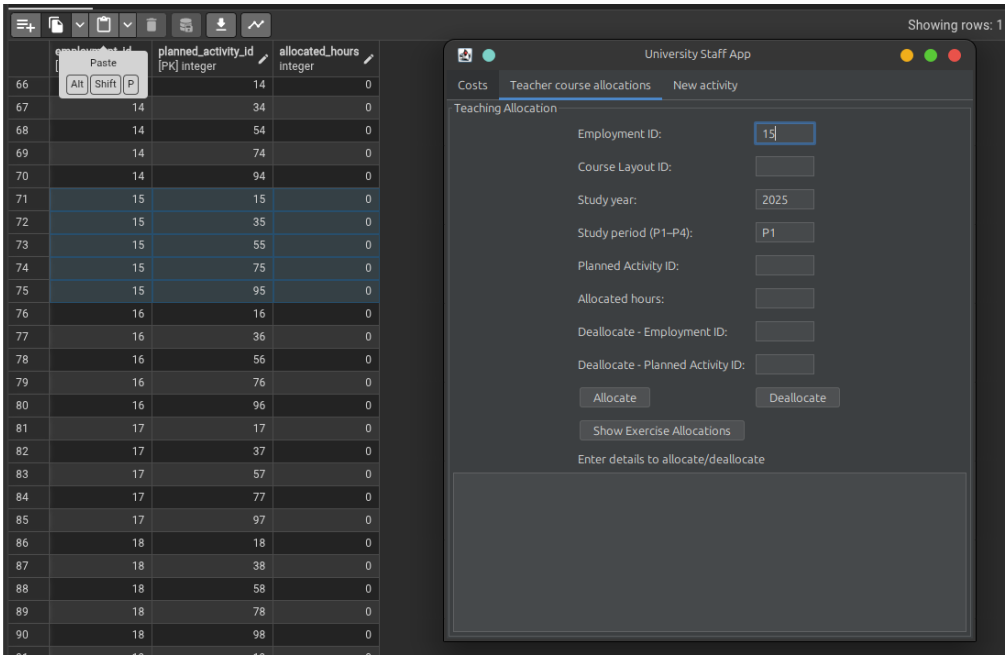


Figure 3: Before course allocation to employee

4.4 Adding a new teaching activity and listing affected allocations

The activity panel allows the user to add a new teaching activity, for example *Exercise*, together with a multiplication factor. The controller passes this request to the

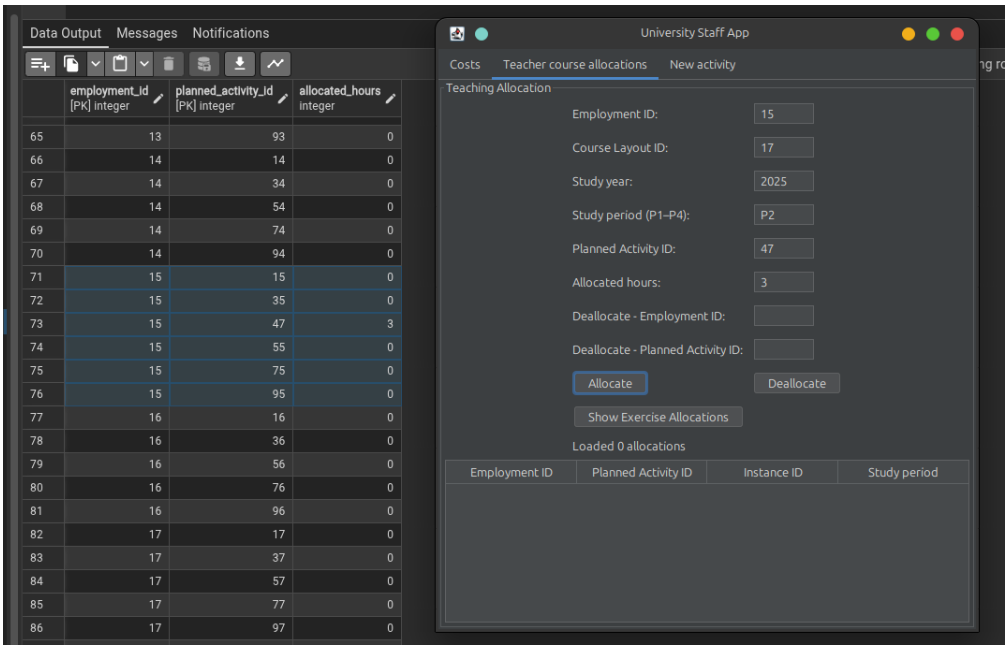


Figure 4: After course allocation to employee

activity service, which inserts a new row into the `teaching_activity` table within an explicit transaction. To show the impact of the new activity, the panel can query all allocations for activities named *Exercise*. The allocation service returns a list of employment IDs, planned activity IDs, course instance IDs, and study periods by joining `employee_activity`, `planned_activity`, `course_instance`, and `teaching_activity`. This output shows both the course instances and the teachers affected by the new activity.

5 Discussion

A central design goal of the application was to respect the ACID properties of database transactions.

Atomicity

The program executes each high-level operation as a single atomic unit of work. Autocommit is explicitly turned off for all connections in the database manager, and the service layer is responsible for starting and ending transactions. For example, increasing the number of registered students and recomputing the course cost is implemented as one transaction in the course service: it reads the current course instance, updates the `num_students` attribute, recomputes planned and actual hours, calculates the new costs, and then commits. If any step fails, the service catches the exception, rolls back

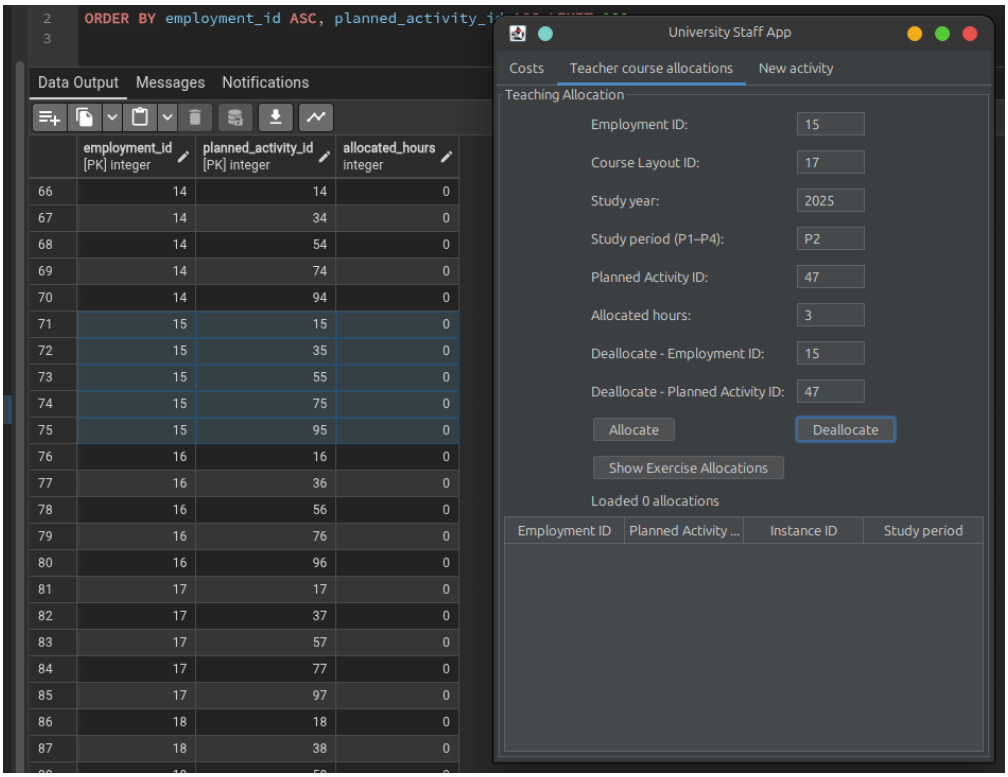


Figure 5: Course deallocation

the transaction, and closes the connection, ensuring that either all steps or none of them are applied.

Isolation

Isolation is handled by using row-level locks in the operations that do read/modify/write. When updating the number of students for a course instance, the course service calls a DAO method that executes `SELECT ...FOR UPDATE` on the target row. This prevents other concurrent transactions from modifying the same instance until the current transaction finishes, and thus avoiding lost updates. Other queries that only read aggregated data, such as computing planned or actual hours or listing allocations, run without explicit locks because they don't write based on the read values and can safely observe a consistent snapshot given by the database.

Durability

Durability is provided by PostgreSQL itself. Once a service method completes successfully and commits its transaction, the updates are guaranteed to be written to stable storage according to the DBMS configuration. If the application crashes after a commit

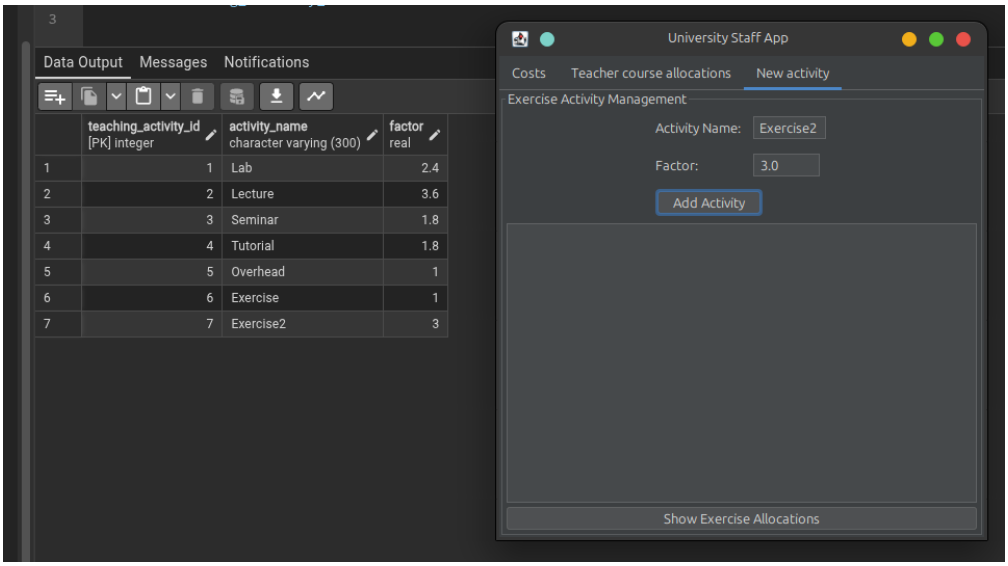


Figure 6: Adding a new teaching activity

returns, the database state will still include all committed allocations, deallocations, added activities, and updates of student counts.

Summary

By turning off auto-commit, delegating all transaction control to the service layer, using `SELECT ...FOR UPDATE` where read–modify–write logic is present, and relying on constraints and triggers for integrity checks, the application adheres to the ACID principles. This design simplifies reasoning about correctness, especially in concurrent scenarios, and makes it easier to extend the system without breaking transactional guarantees.

6 Self Assessment

Naming conventions

Class names, method names, and variables follow established Java naming conventions. Class names such as `CourseService`, `AllocationDAO`, and `ActivityPanel` are nouns, while methods such as `findPlannedHours`, `allocateTeacher`, and `insertTeachingActivity` are verbs that clearly describe their purpose. Database objects use lower case with underscores and meaningful names such as `course_instance`, `employee_activity`, and `v_full_course_workload`, which makes queries self-explanatory.

Transaction handling and auto-commit

Auto-commit is explicitly turned off for all connections by the `DatabaseManager`, and every SQL statement executed through the DAOs runs within an explicit transaction

managed by the service layer. Each service method obtains a connection, performs the necessary DAO calls, and then commits on success or rolls back on failure, using try–catch–finally blocks to ensure that both rollback and connection close are always invoked in the error paths.

Use of SELECT FOR UPDATE

`SELECT ...FOR UPDATE` is used for the course instance selection that participates in a read–modify–write cycle. When increasing the number of students for a course instance and recomputing costs, the service first calls `findInstanceForUpdate`, which locks the corresponding row so that concurrent transactions cannot overwrite each other's updates. Other queries that only read aggregated data, such as computing planned or actual hours, do not use row locks, since they do not perform subsequent updates based on the read values.

Coverage of functional requirements

The program:

- computes planned and actual teaching costs for a chosen course instance using an average teacher salary and the derived hours from the view and allocation table
- supports increasing the number of registered students by 100 for that instance and recomputes the teaching cost accordingly within the same transaction
- can allocate and deallocate teaching loads for teachers, while a trigger ensures that no teacher is responsible for more than four course instances in a given period and year, and the user interface demonstrates both successful and failing allocations
- allows adding a new teaching activity such as *Exercise*, associating it with course layouts and allocations, and querying which course instances and teachers are affected

Higher grade criteria

There is no business logic in the integration layer; the DAOs only provide methods whose purpose is to create, read, update, or delete rows or to compute simple aggregates such as sums of hours. All domain rules [1] are implemented in the service layer. The view and controller layers are also free from business logic: controllers delegate directly to services, and panels only do input validation, formatting, and presentation.

The code's organization mirrors the architectural layers, which makes it relatively easy to navigate and understand. Duplicated code has been systematically removed. Cost calculations are centralized in `CourseService`, allocation logic in `AllocationService`, and each SQL statement appears in only one DAO method. When we need a common pattern (for example transaction handling) it is implemented once per service class rather than repeated throughout the controllers or views.

[1] for example, how to compute planned and actual cost or how to interpret a trigger failure as a teacher overload